Riftbound TCG privé

Rapport d'avancement — Étape 9 : PWA et responsive

09/11/2026

Pour reprendre le projet dans une nouvelle conversation, donner ce PDF (ou le dépôt GitHub) à l'assistant et lui demander de lire **documentation/etat-du-projet.md** puis de continuer à l'étape indiquée comme « prochaine étape ».

Prompt initial (verbatim)

Date : 2026-11-08 — Auteur : Sébastien Girard

> Je souhaite créer un tcg sur Rifbound mais de maniere privé géré par compte et pwd quand tu accèdes au site.
>
> Je souhaite utiliser cet api https://github.com/LouisCourrian/riftbound-cards ce qui me permet de rien stocker dans ma db.
>
> Ce sera hébérger sur cloudeflare gratuitement avec sqlite de cloudflare gratuit.
>
> Il faut pouvoir choisir le pack qu'on veut ouvrir (si c'est vendetta, unleashed ect). pouvoir échanger entre les joueurs et avoir 1 packs toutes les 10 mins. on peut en stocké 15. Avoir des raretés comme dans le vrai jeu de cartes avec les memes taux de drop les memes nbr de cartes par boosters ect.
>
> Faut que ce soit responsible sur tel pc tablette ect. Qu'on puisse utiliser la fct application sur navigateur comme ça c'est plus simple sur pc.
>
> On fait étape par étape. Dans un process de devsecops.
>
> On va utiliser GIthub ça va me permettre de switch sur mon pc portable. Je veux des rapports pdf d'avancement avec le prompt de base sur le fichier comme ça si je perds notre conv tu pourras pile reprendre ou on en est sans etre perdu.

Riftbound TCG privé — État du projet

Ce fichier est la source de vérité pour reprendre le projet. Le rapport PDF de chaque étape est généré depuis ce fichier et depuis `documentation/demande-initiale.md`.

Dernière mise à jour : 2026-11-09

Étape en cours : 9 — PWA et responsive (terminée)

Prochaine étape : 10 — Durcissement sécurité (revue, en-têtes, tests d'abus) et mise en prod. **Pas encore commencée.** Le site est installable (PC, Android, iPhone) et s'ouvre hors ligne (§4sexies).

Dépôt : https://github.com/Crousti-Bats/riftbound-tcg-prive (public, `main` protégée)

Site en production : https://riftbound-tcg.axelgirard.workers.dev

Infrastructure Cloudflare (aucune de ces valeurs n'est secrète)

Élément	Valeur
Compte	`Axelgirard@gmail.com's Account` (ID `14bd3e99e009ec33efebde236223ab6a`)
Worker	`riftbound-tcg` (site + API), URL `riftbound-tcg.axelgirard.workers.dev`
Base D1	`riftbound-tcg`, ID `efbe0899-7a3f-4f63-aad4-4bd9c540eb2d`, région WEUR, nom dans le code `BASE_DE_DONNEES`
Migrations	`base-de-donnees/migrations/NNNN_description.sql`, appliquées automatiquement au déploiement
GitHub	environnement `production` (déploiement depuis `main` uniquement) : secrets `CLOUDFLARE_API_TOKEN` et `CLOUDFLARE_ACCOUNT_ID`, variable `CLOUDFLARE_SOUS_DOMAINE=axelgirard`

0. Reprendre sur un autre PC

```
git clone https://github.com/Crousti-Bats/riftbound-tcg-prive.git
cd riftbound-tcg-prive
npm ci
npm run dev        # site + API en local (http://localhost:5173)
npm test           # tests
```

Pré-requis : Git, Node.js 22 ou plus, GitHub CLI (`gh auth login`), Python avec `pip install reportlab pillow` (rapports PDF et icônes).

Commandes Cloudflare (création d'admin, base distante, journaux) : lancer une fois `npx wrangler login` sur le nouveau PC. Le déploiement, lui, passe par GitHub et ne demande rien.

Pour reprendre avec l'assistant : ouvrir le dossier dans VS Code et dire « on reprend ». Il lit `CLAUDE.md`, puis ce fichier.

Méthode de travail : `git switch -c etape-N`, puis commit, puis `git push`, puis `gh pr create`. On merge quand la CI est verte.

1. Objectif

Un site privé de collection et d'échange de cartes Riftbound, accessible uniquement avec un compte et un mot de passe, hébergé gratuitement sur Cloudflare.

- Ouvrir des boosters en choisissant le set (Origins, Spiritforged, Unleashed, Vendetta, Radiance).
- Composition des boosters et taux de drop alignés sur le vrai jeu.
- Recevoir 1 booster toutes les 10 minutes, avec un stock de 15 au maximum.
- Échanger des cartes entre joueurs.

- Site responsive (téléphone, tablette, PC) et installable comme application (PWA).

2. Décisions d'architecture

Sujet	Décision	Raison
Hébergement	Un seul Cloudflare Worker qui sert le front (Static Assets) et l'API	Offre gratuite, un seul déploiement, aucun CORS
Base de données	Cloudflare D1 (SQLite), offre gratuite	Demandé. Limites : 5 Go, 5 M lectures/jour, 100 k écritures/jour
Données des cartes	`cards.json` de la dernière release de LouisCourrian/riftbound-cards, récupéré par le Worker et mis en cache (Cache API, 24 h)	Aucune carte stockée en base. D1 ne garde que des `cardCode` et des quantités
Images	Liens directs vers le CDN Riot (`imageUrl`)	Rien à héberger
Backend	TypeScript + Hono	Léger, prévu pour Workers
Frontend	React + Vite + TypeScript. PWA avec un service worker écrit à la main (et non `vite-plugin-pwa`, décision de l'étape 9)	PWA installable, responsive, sans dépendance de plus
Authentification	Inscription uniquement sur invitation : l'admin génère des codes. Mots de passe hachés en PBKDF2-SHA256 (WebCrypto), sessions en base avec cookie `HttpOnly; Secure; SameSite=Strict`	Site privé, sans dépendance externe
Aléatoire	`crypto.getRandomValues` côté serveur uniquement	Le client ne peut pas tricher
Timer des boosters	Calcul paresseux : `stock = min(15, stock_saved + floor((now - last_tick)/600 s))`, sans cron	Aucun coût, résultat exact
Échanges	Proposition, puis acceptation. Transfert atomique via `D1 batch()` après vérification de possession	Pas de duplication de cartes
Code source	GitHub (dépôt public, voir §5), synchronisé entre le PC fixe et le portable	Demandé

2bis. Espace admin et monitoring (demandé le 2026-11-08)

Une page `/admin` accessible **uniquement par l'utilisateur** (le propriétaire du site), pour surveiller le site en quasi temps réel.

Contrôle d'accès (défense en profondeur)

- Colonne `role` dans la table des comptes (`joueur` ou `admin`). Le premier compte admin est créé par une commande en ligne réservée à l'utilisateur. Aucun formulaire public ne peut créer d'admin.
- Toutes les routes `/api/admin/*` vérifient le rôle **côté serveur**, à chaque requête. Masquer le menu dans l'interface ne suffit pas.
- Option recommandée à l'étape 10 : protéger aussi `/admin` et `/api/admin/*` avec Cloudflare Access (Zero Trust, gratuit jusqu'à 50 utilisateurs). Il faudrait alors une vérification par e-mail ou par code en plus du mot de passe.
- Chaque action admin est journalisée dans la table `journal_admin` (qui, quoi, quand).

Ce que la page affiche (rafraîchie toutes les 5 à 10 secondes)

- Joueurs connectés récemment (dernière activité de moins de 5 minutes), nombre total de comptes, invitations en attente.
- Activité : boosters ouverts (dernière heure et aujourd'hui, par set), échanges proposés, acceptés ou refusés, dernières cartes rares tirées.
- Sécurité : connexions échouées, comptes ou IP bloqués par la limitation des tentatives, dernières erreurs serveur.
- Santé : état de l'API et de la base, temps de réponse, âge du cache `cards.json`, version déployée.
- Quotas gratuits Cloudflare : requêtes Worker et lectures/écritures D1 du jour, comparées aux limites. Ces chiffres viennent de l'API GraphQL Analytics de Cloudflare, avec un jeton en lecture seule stocké en `wrangler secret`, jamais dans le code.
- Actions : créer ou révoquer une invitation, désactiver un compte, forcer la déconnexion d'un joueur.

Technique : pour démarrer, le navigateur interroge le serveur toutes les quelques secondes (« polling »). C'est simple et ça tient dans l'offre gratuite. On pourra passer à du vrai temps réel (WebSocket via Durable Objects) si le besoin se confirme.

2ter. Authentification (étape 3)

Sujet	Choix
Inscription	Uniquement avec un code d'invitation (256 bits, stocké en SHA-256, usage unique, expiration de 24 h à 7 jours, pseudo réservable). Le lien `/inscription#code=…` place le code après le `#`, qui n'est jamais envoyé au serveur
Admin	Une seule façon d'en créer un : `npm run admin:invitation -- --pseudo X --production`, qui demande d'être connecté à Cloudflare. Le site ne peut créer que des invitations `joueur`. L'utilisateur a un seul compte `Crousti` (joueur et admin)
Accès admin	Rôle vérifié côté serveur à chaque requête, avec re-saisie du mot de passe (élévation) valable 15 minutes glissantes et un bouton « Fermer l'accès admin »
Mots de passe	6 à 128 caractères (minimum abaissé de 12 à 6 à la demande de l'utilisateur le 2026-11-08 ; risque accepté : un mot de passe court est vite cassé si la base fuit, mais la limitation des tentatives protège le site), PBKDF2-SHA256 à 100 000 itérations (maximum Workers), sel aléatoire, format versionné. **À surveiller** : le plan gratuit limite le CPU à 10 ms par requête, mais Cloudflare tolère des pics. Si les connexions échouent en production (erreur 1102), baisser les itérations (le format le permet)
Sessions	Jeton de 256 bits dans un cookie `__Host-session` (HttpOnly, Secure, SameSite=Strict). Seule l'empreinte SHA-256 est en base. Durée de 7 jours glissants, 30 jours maximum. La déconnexion et la désactivation du compte suppriment les sessions
Anti force brute	Connexion : 5 échecs par pseudo ou 20 par IP en 15 minutes, puis 429. Inscription : 10 échecs par IP et par heure. Élévation : 5 échecs en 15 minutes
Énumération	Même message et même temps de calcul (hash factice) pour un pseudo inconnu ou un mauvais mot de passe
CSRF	SameSite=Strict, en-tête `Origin` obligatoire et identique au site pour POST, PUT, PATCH et DELETE, et JSON obligatoire
Entrées	Zod en mode strict : les champs inconnus sont refusés (par exemple, on ne peut pas envoyer `role: "admin"`). Corps limité à 16 Ko

Sujet	Choix
SQL	Uniquement `.prepare("… ? …").bind()`. La règle ESLint `no-restricted-syntax` fait échouer la CI sinon, et elle est elle-même testée
Journal	Table `journal` : inscriptions, connexions réussies, échouées et bloquées, élévations, actions admin

3. Données de cartes (analysées le 2026-11-08, release v2026-11-04)

1407 cartes. Raretés présentes dans les données : Common, Uncommon, Rare, Epic, Showcase.

Set	Code	Ouvrable en booster
Origins	ogn	oui
Proving Grounds	ogs	non (produit starter)
Spiritforged	sfd	oui
Unleashed	unl	oui
Vendetta	ven	oui
Radiance	rad	oui

On classe les variantes à partir de `cardNumber` :

- `NNN-TTT` avec NNN ≤ TTT : carte de base, avec sa rareté.
- `NNNa-TTT` : Alt Art (Showcase).
- `NNN-TTT` avec NNN > TTT : Overnumbered (Showcase).
- `NNN-star-TTT` : Overnumbered signée (Showcase).
- `spX-NNN` : Special Alt (Showcase).
- `rNN` et `rNNa` : Rune et Rune alt. `tNN` : Token ou Battlefield token.

3bis. Catalogue (étape 4)

- **Source :** le miroir Hugging Face officiel de l'auteur `https://huggingface.co/datasets/Wysme/riftbound-cards/resolve/main/cards.json` en premier, avec la release GitHub `…/releases/latest/download/cards.json` en secours. C'est le même fichier d'environ 1,5 Mo, mis à jour chaque jour par l'auteur. Chaque source a 8 s au plus. Si les deux échouent et que rien n'est en cache, l'API répond 503 `catalogue_indisponible`. Rien n'est stocké en base.
- **Cache :** mémoire du Worker, puis Cache API Cloudflare (catalogue déjà transformé, 24 h), puis téléchargement. Si la source tombe en panne, on garde l'ancienne version en mémoire. Un seul téléchargement est lancé même si plusieurs requêtes arrivent en même temps.
- **Validation :** chaque carte de la source est validée par Zod (format du code, rareté connue, longueurs maximales, image venant obligatoirement de `https://cmsassets.rgpub.io/`). Une carte invalide est ignorée et comptée, jamais affichée.
- **Variante** (calculée à partir de `cardNumber`, voir `serveur-api/catalogue/classification-cartes.ts`) : `base`, `alt` (NNNa), `overnumbered` (NNN > total), `signee` (NNN-star), `special` (spX), `ultime` (overnumbered de rareté Epic : Baron Nashor `unl-238-219`), `rune`, `rune_alt`, `token`.
- **Vérification du 2026-11-08 :** 1 407 cartes, 0 ignorée. Dans chaque set, le nombre de cartes `base` correspond au total imprimé (ogn 298, sfd 220, unl 219, ven 166, rad 167).
- **Attention pour l'étape 5 :** les runes d'Origins sont numérotées comme des cartes normales (variante `base`, `cardType` = `Rune`). Le slot Rune doit donc sélectionner sur `type === "Rune"`, pas seulement sur la variante. Spiritforged et Unleashed n'ont pas de rune dans la source, seulement des tokens.

- **Langue (décision de l'utilisateur du 2026-11-08) :** noms, textes et images des cartes restent en **anglais**. Riot ne publie pas les cartes françaises en ligne : la galerie `playriftbound.com/fr-fr` ne traduit que les libellés et affiche les mêmes images anglaises. Le site reste en français.
- **Complétude (comparaison avec la galerie officielle le 2026-11-08) :** 1 408 cartes officielles, 1 407 dans la source. Il manque `ogn-189-000` (Kayn, Unleashed, promo « /000 ») et `sfd-054-221` (Jax, version Rare ; seule l'Alt Art `sfd-054a-221` est présente). C'est un défaut de la source, à signaler éventuellement à son auteur.
- **Étiquettes et filtre par type (2026-11-09) :** le champ source `cardTypeLabels` est lu dans `etiquettes` (sans le type principal) : `Champion` (352 cartes : unités et légendes), `Signature` (60), `Token` (14 en tout), `Basic` (runes). Le filtre `type` accepte le type principal **ou** une étiquette (`correspondAuType`), d'où les options Champion et Signature. Clé du Cache API passée en `catalogue-v3`.
- **Images de secours :** composant `application-web/composants/image-carte.tsx`. Si la miniature redimensionnée du CDN échoue (cas vu sur Ahri `sfd-227-221` : image vide jusqu'au clic), il essaie la taille 744, puis l'image d'origine.
- **Orientation :** les 77 Battlefields sont des images horizontales (1039×744). Le champ `orientation` (`portrait` ou `paysage`) est déduit des dimensions présentes dans l'URL. Dans la grille, ils sont tournés de 90° dans le cadre vertical ; dans la fiche, ils sont affichés à l'horizontale. Clé du Cache API passée en `catalogue-v2` (à incrémenter à chaque changement du format).
- **Images :** le CDN Riot redimensionne. `?w=300&fm=webp` donne environ 28 Ko pour les miniatures, au lieu de 1,2 Mo pour le PNG d'origine, et `?w=744&fm=webp` sert pour la fiche.
- **API (connexion obligatoire) :** `GET /api/catalogue/sets`, `GET /api/catalogue/cartes?set=&rarete=&variante=&type=&recherche=&page=` (60 par page, recherche sans accents ni majuscules), `GET /api/catalogue/cartes/:code`.
- **À surveiller :** le plan gratuit limite le CPU à 10 ms par requête. Le premier chargement (lecture du JSON de 1,5 Mo) peut dépasser cette limite. S'il y a des erreurs 1102 sur le catalogue en production, ajouter un stockage KV du catalogue transformé.

4. Composition d'un booster (14 cartes, source officielle Riot)

Slot	Nb	Contenu
Common	7	Common
Uncommon	3	Uncommon
Foil	1	Common, Uncommon, Rare ou Epic en foil
Rare+	2	Rare, Epic, Alt Art, Special Alt, Overnumbered, Signée
Rune / Token	1	Rune, Rune alt ou Token

Mise à jour du 2026-11-09 : tous les taux Rare+ sont divisés par 2 à la demande de l'utilisateur. Par booster : Epic 1/12, Alt Art 1/24, Overnumbered 1/144, Signée 1/480, Special et Ultime 1/2000. Par slot Rare+ : Epic 3,667 % ((1/12 − 0,01)/2), Alt 2,08 %, Over 0,347 %, Signée 0,104 %, Special et Ultime 0,025 %, Rare ≈ 93,77 %. Le slot Foil est inchangé. Les tableaux ci-dessous sont l'historique des taux d'origine ; la valeur en vigueur est dans `serveur-api/boosters/configuration-taux.ts`.

Taux de référence, validés par l'utilisateur le 2026-11-08 (ils font foi, à coder dans un fichier de configuration à l'étape 5) :

Rareté	Fréquence par booster	Soit par display de 24
Epic (orange)	1 booster sur 6	4
Alt Art	1 booster sur 12	2
Overnumbered	1 booster sur 72	1 pour 3 displays
Overnumbered signée	1 booster sur 240	1 pour 10 displays

On les convertit en probabilités pour chacun des 2 slots Rare+. Comme il y a 2 slots, « 1 pour N boosters » devient 1/(2N) par slot. Pour l'Epic, on retire d'abord la part du slot Foil (1 %), pour que le total observé reste de 1 sur 6 :

Résultat	Proba par slot Rare+	Calcul
Epic	7,83 %	(1/6 − 0,01) / 2
Alt Art	4,17 %	1/24
Overnumbered	0,694 %	1/144
Overnumbered signée	0,208 %	1/480
Special Alt (sp)	0,05 %	1 booster sur 1 000, soit 1/2000 par slot (validé par l'utilisateur le 2026-11-08)
Ultime (Baron Nashor, Unleashed uniquement)	0,05 %	1 booster sur 1 000, soit 1/2000 par slot (validé le 2026-11-08)
Rare	87,00 %	le reste. Pour un set sans Special ni Ultime, leur part revient aux Rares

Les tests de l'étape 5 simuleront au moins 100 000 boosters et vérifieront ces fréquences **par booster** : Epic environ 16,7 %, Alt environ 8,3 %, Over environ 1,39 %, signée environ 0,42 %, Special et Ultime environ 0,1 %.

Autres décisions pour l'étape 5 (validées par l'utilisateur le 2026-11-08) :

- Un nouveau joueur reçoit **15 boosters** à l'inscription (stock plein). Ensuite, +1 toutes les 10 minutes, 15 au maximum.
- Le slot Rune sélectionne sur `type === "Rune"` (les runes d'Origins sont en variante `base`). Pour un set sans rune (Spiritforged, Unleashed), on tire un Token du set, sinon une Rune d'Origins (voir §3bis).
- **Décisions tranchées à l'étape 5 (par l'utilisateur le 2026-11-09) :**
- Les Légendes (type Legend, Rare) et les Terrains (type Battlefield, Uncommon) **entrent** dans leurs pools de rareté normaux.
- Les **doublons** à l'intérieur d'un même booster sont **autorisés** (tirages indépendants, statistiquement corrects).
- Les cartes **foil** sont stockées **séparément** des normales dans la collection (clé `compte_id, card_code, est_foil`).

Slot Foil (estimation communautaire) : 75 % Common, 20 % Uncommon, 4 % Rare, 1 % Epic.

Slot Rune : 92 % Rune, 8 % Rune alt. Si le set n'a pas de rune, on tire un Token du set ou une Rune d'Origins.

4bis. Moteur de boosters (étape 5)

- **Taux :** une seule source de vérité, `serveur-api/boosters/configuration-taux.ts` (composition, probabilités Rare+, Foil, Rune, stock). Tout changement de taux se fait là.
- **Tirage :** `serveur-api/boosters/tirage-booster.ts` est une **fonction pure** (pools + set + aléatoire → 14 cartes), donc testable à fond. L'aléatoire est injectable (`aleatoire.ts`) : `crypto.getRandomValues` en production (côté serveur, non prédictible), PRNG seedé déterministe pour les tests (pas de test instable). Si le pool d'une catégorie Rare+ est vide pour le set, sa probabilité est reversée aux Rares. Slot Rune : rune du set (ou token, ou repli sur une rune d'Origins).
- **Pools :** `serveur-api/boosters/pools.ts` pré-indexe les cartes par set/rareté/variante/type, mémoïsé sur `catalogue.chargeLe`. Runes et Tokens exclus des pools de rareté ; Légendes et Terrains inclus (décision du 2026-11-09).
- **Stock paresseux :** `serveur-api/boosters/stock.ts` — `stock = min(15, stockSauve + floor((now − tick)/10 min))`, sans cron. 15 à l'inscription. Au plafond le minuteur s'arrête ; sinon la progression partielle est préservée.

- **API (connexion obligatoire) :** `GET /api/boosters/etat` (stock, prochain booster, sets ouvrables) ; `POST /api/boosters/ouvrir {set}` (consomme 1 booster, tire 14 cartes, met à jour la collection, renvoie les cartes enrichies). 503 `boosters_indisponibles` si le catalogue est injoignable.
- **Atomicité / anti-triche :** le décrément du stock est protégé par **concurrence optimiste** (`UPDATE … WHERE stock = ? AND tick = ?`) ; on n'ajoute les cartes (un seul `batch()` : 14 upserts + log d'ouverture) qu'après un décrément confirmé. Deux ouvertures simultanées ne peuvent pas dépenser un seul booster deux fois (la 2e reçoit 409 `conflit_ouverture`), et aucune carte n'est jamais ajoutée sans consommer de booster. Colonne `stock_boosters` avec `CHECK (0 ≤ stock ≤ 15)` en défense.
- **Base (migration `0003`) :** colonnes `stock_boosters` / `boosters_tick` sur `comptes` ; table `collection` (`compte_id, card_code, est_foil, quantite`) — **jamais** de donnée de carte, seulement des `cardCode` et des quantités ; table `ouvertures_boosters` (pour le monitoring de l'étape 8).
- **Tests :** `serveur-api/tests/boosters.test.ts` — stock, pools, structure, **100 000 boosters seedés** (fréquences par booster vérifiées : Epic ≈16,7 %, Alt ≈8,3 %, Over ≈1,39 %, signée ≈0,42 %, Special/Ultime ≈0,1 % ; Foil 75/20/4/1 ; Rune 92/8), redistribution des taux pour un set sans Special/Ultime, et API (15 au départ, ouverture, collection, foil distinct, 409/400/401, régénération dans le temps). Fixture `tests/donnees/` via `catalogue-de-test.ts`.

4ter. Interface boosters et collection (étape 6)

- **Hub d'ouverture (accueil) :** `application-web/composants/panneau-boosters.tsx` affiche le stock `N/15`, un **minuteur** du prochain booster (décompte client, relecture de `/etat` à zéro), le choix du set ouvrable et le bouton Ouvrir.
- **Animation :** `composants/ouverture-booster.tsx` révèle les 14 cartes **en cascade** (`animation-delay` par carte), avec **halo** sur les rares+ et **voile brillant** sur les foil ; clic = fiche. Respecte `prefers-reduced-motion`.
- **Page Collection (`/collection`) :** `pages/page-collection.tsx` — filtres (set, type, rareté, variante, recherche) + bascules **foil** / **doublons**, grille avec badge quantité (×N) et foil (□), fiche au clic, pagination, en-tête de **complétion par set** (cartes de base possédées / total).
- **Composants partagés :** `composants/fiche-carte.tsx`, `filtres-cartes.tsx`, `pagination.tsx` (extraits du catalogue, réutilisés par catalogue, collection et ouverture).
- **API collection (lecture, connexion obligatoire) :** `GET /api/collection?set=&rarete=&variante=&type=&recherche=&foil=&doublons=&page=` (cartes possédées enrichies depuis le catalogue, 60/page) et `GET /api/collection/resume` (totaux + complétion par set). Aucune donnée de carte en base. `sansAccents` est partagé via `serveur-api/catalogue/recherche-texte.ts`.
- **Vérifié bout-en-bout** le 2026-11-09 contre le vrai runtime local (miniflare D1 + vraie source) : inscription → 15 boosters → ouverture (14 cartes, composition exacte) → collection et complétion (ogn 298 cartes de base). 96 tests automatiques (90 + 6 pour la collection).

4quater. Échanges entre joueurs (étape 7)

Règles décidées par l'utilisateur le 2026-11-09 : plusieurs cartes contre plusieurs, **dons autorisés** (un côté peut être vide, pas les deux), on peut parcourir **toute la collection** des autres joueurs, une proposition expire au bout de **48 heures**. Limites anti-abus (`serveur-api/echanges/regles-echanges.ts`) : 20 cartes différentes et 60 exemplaires maximum par côté, 20 propositions en attente maximum par joueur, message de 200 caractères maximum.

- **Base (migration `0004`) :** table `echanges` (auteur, destinataire, statut `en_attente | accepte | refuse | annule | expire | echoue`, message, dates, `jeton_acceptation`) et table `echanges_cartes` (`sens` = `donne` de l'auteur vers le destinataire, ou `demande` dans l'autre sens ; `card_code`, `est_foil`, `quantite`). Seulement des cardCode et des quantités.
- **Pas de réservation des cartes** pendant l'attente : la possession est vérifiée à la création, puis **revérifiée à l'acceptation**.

- **Acceptation atomique :** un seul `batch()` D1 (une transaction). (1) Réservation `UPDATE … SET statut='accepte', jeton_acceptation=? WHERE statut='en_attente' AND destinataire_id=? AND expire_le > now`. (2) Pour chaque ligne : retrait chez le donneur (ligne créée à 0 si absente, puis `quantite - n`) et ajout chez le receveur. Chaque transfert porte la garde `EXISTS (… jeton_acceptation = ?)` : il ne s'exécute que si la réservation a réussi **dans ce lot**. (3) Suppression des lignes à 0. S'il manque une carte, `CHECK (quantite >= 0)` fait échouer le lot et **tout est annulé** : l'échange passe en `echoue` et l'API renvoie 409 `cartes_plus_disponibles`. Une double acceptation, ou deux acceptations simultanées, n'exécutent l'échange qu'une fois.
- **Expiration paresseuse :** les propositions dépassées passent en `expire` à la lecture, sans tâche planifiée.
- **API (connexion obligatoire) :** `GET /api/echanges/joueurs`, `POST /api/echanges` (proposer), `GET /api/echanges` (reçues, envoyées, historique, cartes enrichies depuis le catalogue), `GET /api/echanges/resume` (pastille), `POST /api/echanges/:id/accepter | refuser | annuler`. Seul le destinataire accepte ou refuse, seul l'auteur annule. Collection d'un autre joueur : `GET /api/collection/joueur/:pseudo` (mêmes filtres que la sienne).
- **Interface :** bouton « Échanges » sur l'accueil avec une pastille du nombre de propositions reçues. Page `/echanges` (onglets Reçues, Envoyées, Historique ; Accepter, Refuser, Annuler ; temps restant). Page `/echanges/nouveau` (choix du joueur, collection de l'autre et la mienne avec filtres, boutons « + Normale » / « + Foil », panier modifiable −/+, message).
- **Tests :** `serveur-api/tests/echanges.test.ts` (17 tests) : création, possession (foil distinct), validation, limite de propositions, échange dans les deux sens, don, droits, double et triple acceptation simultanée, **atomicité** (carte disparue entre-temps : rien ne bouge), expiration à 48 h, refus et annulation, liste des joueurs, collection des autres. Vérifié aussi de bout en bout sur le vrai moteur D1 local (échange réussi, et annulation complète quand une carte manque).

4quinquies. Monitoring admin (étape 8)

- **Route :** `GET /api/admin/monitoring` (`serveur-api/administration/monitoring.ts`), réservée à l'admin avec accès ouvert. Elle **ne prolonge pas** l'accès admin (`exigerAdminSansProlonger`) : sinon une page laissée ouverte garderait l'accès indéfiniment, avec une écriture D1 toutes les 8 s. Réponse `Cache-Control: no-store`. Lecture seule : les 11 requêtes préparées partent en un seul `batch()` D1.
- **Contenu :** joueurs (en ligne si activité < 5 min, comptes, désactivés, invitations en attente, joueurs actifs sur 24 h) ; activité (boosters sur 1 h et 24 h, par set ; échanges sur 24 h : proposés, acceptés, refusés, échoués, en attente ; 15 dernières cartes remarquables) ; sécurité (connexions échouées et bloquées, inscriptions refusées, élévations refusées sur 1 h et 24 h, blocages actifs de la limitation des tentatives, erreurs serveur) ; santé (API, base et sa latence, âge du catalogue, version déployée) ; quotas Cloudflare du jour.
- **« En ligne » :** la dernière activité est maintenant écrite au plus toutes les **2 minutes** (au lieu de 5), pour que le seuil de 5 minutes soit fiable.
- **Base (migration `0005`) :** `tirages_remarquables` (cardCode d'une carte Epic, Alt Art, Overnumbered, signée, Special ou Ultime, écrit dans le même lot atomique que l'ouverture) et `erreurs_serveur` (méthode, chemin sans paramètres, message tronqué à 300 caractères ; jamais le corps ni les en-têtes ; purge à 7 jours, `app.onError` via `waitUntil`). Index sur `journal(type, horodatage)` et `echanges(cree_le)`, `echanges(termine_le)`.
- **Quotas Cloudflare :** `serveur-api/administration/quotas-cloudflare.ts` interroge l'API GraphQL Analytics (`workersInvocationsAdaptive` pour les requêtes Worker, `d1AnalyticsAdaptiveGroups` pour les lignes D1 lues et écrites), depuis 00:00 UTC (remise à zéro des quotas gratuits). Limites affichées : 100 000 requêtes, 5 M lignes lues, 100 000 lignes écrites par jour. Résultat gardé 60 s en mémoire. L'ID du compte est en `vars` dans wrangler (non secret). Le jeton est un **secret Worker** `JETON_ANALYTICS_CLOUDFLARE` (lecture seule) : il n'est jamais dans le code ni renvoyé au navigateur. Sans jeton, le tableau affiche « non configuré ».
- **Version déployée :** binding `version_metadata` (`VERSION_DEPLOYEE` : id et date du déploiement).

- **Interface :** onglet « Monitoring » (par défaut) de `/admin`, `application-web/composants/tableau-monitoring.tsx`. Rafraîchi toutes les 8 s, en pause quand l'onglet du navigateur est caché, avec un bouton Pause. Jauges de quotas (vert, puis orange à 70 %, puis rouge à 90 %). Les actions (invitations, désactivation, déconnexion forcée) restent dans les onglets existants.
- **Tests :** `serveur-api/tests/monitoring.test.ts` (8 tests) : contrôle d'accès, non-prolongation de l'accès admin, compteurs, blocages, traçage exact des tirages remarquables, erreur 500 enregistrée sans détail renvoyé au client, quotas GraphQL (sommes, jeton en en-tête et jamais renvoyé, cache de 60 s, erreurs HTTP, GraphQL, réseau et format). Vérifié aussi de bout en bout sur le vrai runtime local (miniflare et D1).

4sexies. PWA et responsive (étape 9)

- **Choix technique :** un service worker **écrit à la main** (≈ 100 lignes) au lieu de `vite-plugin-pwa` prévu au §2. Raisons : aucune dépendance de plus (Workbox et ses dizaines de paquets = surface d'attaque de la chaîne d'approvisionnement), compatibilité garantie avec Vite 8 et le plugin Cloudflare, et chaque règle de cache est lisible et testée.
- **Fabrication :** le modèle `application-web/service-worker/modele-service-worker.js` est transformé au build par `configuration/plugin-service-worker.ts` en `dist/client/service-worker.js`. Le plugin y injecte la liste des fichiers à précharger (coquille `/`, manifeste, favicon, icône 192, et le JS et le CSS du build) et une **version** = empreinte SHA-256 de cette liste. Chaque déploiement qui change le site change donc la version : le nouveau service worker s'installe, s'active tout de suite (`skipWaiting` + `clients.claim`) et supprime les anciens caches `riftbound-*`. Pas de service worker en développement (`npm run dev`).
- **Règles de cache :** `/api/*` n'est **jamais** intercepté ni mis en cache : aucune donnée de compte (collection, échanges, session) ne reste sur l'appareil, rien à effacer à la déconnexion, et le serveur reste seul juge (anti-triche). Requêtes non GET et autres sites (images du CDN Riot) non interceptés non plus (le cache HTTP du navigateur suffit pour les images ; mettre en cache des réponses « opaques » coûterait ~7 Mo de quota chacune dans Chrome). Pages : réseau d'abord, coquille `index.html` en secours sans réseau. `/assets/*` (noms avec empreinte, immuables) : cache d'abord. Autres fichiers du site : réseau d'abord. Une erreur serveur (500) ne remplace jamais la coquille en cache.
- **Hors-ligne partiel :** l'application s'ouvre sans réseau (depuis l'icône ou le navigateur), affiche un **bandeau « Hors ligne »** et un message clair avec un bouton Réessayer. Elle se reconnecte **toute seule** au retour du réseau, puis réessaie toutes les 15 s tant que le serveur ne répond pas. Une coupure pendant l'utilisation donne le message « Pas de connexion Internet » au lieu d'une erreur technique (`appels-api.ts`, codes `hors_ligne` et `serveur_injoignable`).
- **Installation :** `fichiers-statiques/manifest.webmanifest` (nom, `display: standalone`, couleurs du site, icônes 192/512 et une icône **maskable** pour Android, raccourcis Ma collection et Échanges). Carte « Installer l'application » sur l'accueil : bouton natif sur Chrome, Edge et Android (`beforeinstallprompt`), mode d'emploi sur iPhone et iPad (Partager → « Sur l'écran d'accueil »). Bouton « Plus tard » mémorisé dans le navigateur. Masquée si l'application est déjà installée.
- **En-têtes :** `service-worker.js` et le manifeste en `Cache-Control: no-cache` (mises à jour immédiates). CSP complétée par `manifest-src 'self'; worker-src 'self'`.
- **Responsive :** `viewport-fit=cover` + marges `safe-area` (encoche, barre d'accueil de l'iPhone) ; `100dvh` pour les fenêtres (les barres du navigateur mobile ne coupent plus le bas) ; très petits écrans ≤ 380 px (boutons pleine largeur, grilles et filtres resserrés) ; pas d'effet de survol « collé » sur écran tactile ; pas de sélection de texte sur les boutons dans l'application installée.
- **Tests :** `serveur-api/tests/pwa.test.ts` (17 tests). Le service worker généré est exécuté dans un bac à sable `node:vm` avec de faux caches et un faux réseau : préchargement, activation et nettoyage des anciennes versions, **jamais l'API ni les POST ni les autres sites**, pages hors ligne, erreur 500, cache d'abord sur `/assets/`, 503 propre. Plus le manifeste (dimensions réelles des PNG lues dans le fichier), `index.html` et `_headers`. Test de mutation fait : retirer l'exclusion de `/api/` fait bien échouer le test de sécurité.

- **Vérifié dans un vrai navigateur** (Edge headless piloté par CDP, build de production servi en local) : aucun défilement horizontal sur les 6 pages à 360, 820 et 1366 px, ni pendant l'ouverture d'un booster ; service worker actif et contrôlant la page ; manifeste sans erreur ; **Edge déclare le site installable** ; serveur arrêté → l'application s'ouvre quand même et affiche « Hors ligne » ; serveur relancé → retour automatique sur la page sans clic ; mise à jour du build → nouveau cache, ancien supprimé.
- **Correctif de test au passage :** `echanges.test.ts` « proposition expirée » avançait l'horloge de 48 h, ce qui périmait le catalogue en mémoire (24 h) et déclenchait un **vrai téléchargement** : test instable (échec à 5 s quand le réseau est lent). Le catalogue de test est maintenant rechargé après le saut d'horloge.

5. Démarche DevSecOps

- Le dépôt est **public** (choix du 2026-11-08). Sur le plan gratuit, c'est la seule façon d'avoir la protection de branche, CodeQL et la push protection. Le site reste privé grâce à l'authentification.
- Branche `main` protégée par un ruleset « Protection main » : PR obligatoire (0 approbation, on est seul), pas de force-push ni de suppression, merge en squash uniquement, et 4 contrôles requis : `Lint, types, tests, build`, `Audit des dépendances`, `CodeQL (SAST)`, `Gitleaks (secrets)`.
- `.github/workflows/verifications-qualite.yml` lance le lint (ESLint), le typecheck (tsc), les tests (Vitest), le build, `npm audit` et la dependency review sur les PR. Il vérifie aussi que `serveur-api/types-cloudflare-generes.d.ts` est à jour.
- `.github/workflows/analyses-securite.yml` lance CodeQL (`security-extended`, JS/TS et Actions) et Gitleaks, à chaque PR, à chaque push et chaque lundi.
- Dependabot (npm et Actions, chaque semaine), alertes et correctifs automatiques activés. Secret scanning et push protection activés. Signalement privé de vulnérabilités activé (voir `.github/SECURITY.md`).
- Chaîne d'approvisionnement : versions npm exactes (`.npmrc save-exact`), lockfile, `npm ci`, actions épinglées par SHA, `permissions` minimales.
- Override `minimatch@11.0.5` dans `package.json` : corrige GHSA-g3hv-75wg-m249 (ReDoS, moderate), remonté via eslint 10. On pourra le retirer quand eslint le corrigera.
- Les secrets (token Cloudflare) sont uniquement dans GitHub Secrets. Jamais dans le code.
- En-têtes de sécurité : CSP stricte, HSTS, X-Frame-Options DENY, Referrer-Policy.
- Rate limiting sur la connexion. Validation des entrées avec Zod. Requêtes SQL paramétrées uniquement.
- Le déploiement en production se fait automatiquement au merge sur `main`, via `.github/workflows/deploiement-production.yml`. Le workflow lance les tests, le build, les migrations D1 `--remote`, puis `wrangler deploy` et vérifie `/api/health`. Il utilise l'environnement GitHub `production`, limité à la branche `main`. Le jeton Cloudflare n'a que les droits Workers et D1.
- `/api/health` vérifie aussi la base (`SELECT 1`). Si la base est en panne, il répond 503 sans détail interne.
- Dependabot ignore les versions majeures de TypeScript : TS 7 n'est pas supporté par typescript-eslint (PR #2 fermée).

6. Feuille de route

#	Étape	Statut
0	Cadrage, architecture, analyse des données, système de rapports	Terminé
1	Outils (Git, Node), dépôt GitHub, squelette du monorepo, CI sécurité	Terminé
2	Compte Cloudflare, Worker et D1, premier déploiement "Hello" via la CI	Terminé

#	Étape	Statut
3	Authentification : invitations, connexion, sessions, limitation des tentatives. **Rôle admin** (compte de l'utilisateur uniquement), première page `/admin` pour gérer les invitations et les comptes. Règle ESLint qui interdit le SQL non préparé	Terminé
4	Catalogue des cartes (fetch et cache du JSON, API sets et cartes)	Terminé
5	Moteur de boosters (taux, tirage, tests statistiques) et timer 10 min / 15 max	Terminé
6	Frontend : ouverture de booster animée, collection, filtres	Terminé
7	Échanges entre joueurs	Terminé
8	**Admin : monitoring en quasi temps réel** (voir §2bis et §4quinquies)	Terminé
9	PWA (installable, hors-ligne partiel), responsive (voir §4sexies)	Terminé
10	Durcissement sécurité (revue, en-têtes, tests d'abus) et mise en prod	À faire

7. Actions manuelles en attente (utilisateur)

1. **Quotas Cloudflare dans le monitoring :** créer un jeton sur https://dash.cloudflare.com/profile/api-tokens → « Create Custom Token », permission **Account** → **Account Analytics** → **Read**, limité au compte `Axelgirard@gmail.com's Account`. Puis, depuis le dossier du projet : `npx wrangler secret put JETON_ANALYTICS_CLOUDFLARE --config configuration/cloudflare-wrangler.jsonc` et coller le jeton. Ne jamais le mettre dans le code ni dans GitHub.
2. Sur le portable : installer Git, Node.js LTS et GitHub CLI (`winget install --id Git.Git -e`, `OpenJS.NodeJS.LTS`, `GitHub.cli`), puis lancer `gh auth login`.
3. **Après le déploiement de l'étape 9 :** installer l'application sur ton téléphone et ton PC pour valider en vrai (Android/PC : carte « Installer l'application » de l'accueil ; iPhone : Safari → Partager → « Sur l'écran d'accueil »), puis passer en mode avion et rouvrir l'application : elle doit s'ouvrir et afficher « Hors ligne ».
4. **Brouillon à trancher (PC fixe uniquement) :** un travail non commité trouvé sur le PC fixe le 2026-11-09 (wishlist `/api/wishlist`, liste `/api/joueurs`, refactor `collection/enrichir.ts`, migration nommée `0004_wishlist.sql` qui entre en conflit avec `0004_echanges.sql`) a été mis de côté dans `git stash` (« brouillon wishlist+joueurs »). Rien n'est perdu : `git stash list` pour le voir. À reprendre plus tard (avec une migration renumérotée) ou à supprimer (`git stash drop`).

8. Journal

- **2026-11-08 — Étape 0.** Analyse de l'API (1407 cartes, 6 sets), recherche des taux officiels, choix de l'architecture, mise en place des rapports PDF (`outils/generer-rapport-pdf.py`).
- **2026-11-08 — Étape 1.** Dépôt public `Crousti-Bats/riftbound-tcg-prive` créé. Squelette : Worker Hono (`worker/`, route `/api/health`, en-têtes de sécurité, 3 tests), front React 19 et Vite 8 (`src/`), `@cloudflare/vite-plugin`, TypeScript strict, ESLint. CI et sécurité (CodeQL, Gitleaks, audit, dependency review, Dependabot), ruleset sur `main`. Le lint, les types, les tests et le build passent en local, avec 0 vulnérabilité.
- **2026-11-08 — Réorganisation.** Les fichiers sont rangés dans des dossiers par fonction, avec des noms explicites en français (voir ci-dessous). Les anciens chemins (`worker/`, `src/`, `docs/`, `tools/`) n'existent plus. Les rapports PDF 00 et 01 citent encore ces anciens chemins, c'est normal : ce sont des archives.

- **2026-11-08 — Étape 2.** `wrangler login` effectué sur le PC fixe. Base D1 `riftbound-tcg` créée (WEUR), migration `0001_initialisation.sql` (table `parametres`) appliquée en local et en production. Premier déploiement manuel réussi : https://riftbound-tcg.axelgirard.workers.dev/api/health répond `{"status":"ok","base":"ok"}`, avec les en-têtes CSP, HSTS et X-Frame-Options vérifiés. Workflow de déploiement automatique ajouté, environnement GitHub `production` créé (branche `main` uniquement). L'assistant n'a pas le droit d'écrire des secrets GitHub ni de fermer des PR : ces actions reviennent à l'utilisateur (§7). 4 tests passent.
- **2026-11-08 — Admin.** L'utilisateur demande une page admin de monitoring en quasi temps réel, réservée à lui seul. Ajout du §2bis. La feuille de route passe à 11 étapes : rôle admin à l'étape 3, monitoring à l'étape 8. Étape 3 : ajout d'une règle ESLint contre le SQL non préparé.
- **2026-11-08 — Taux de drop.** L'utilisateur a fourni les taux de référence : Epic 1/6, Alt 1/12, Over 1/72, Over signée 1/240 boosters. Ils remplacent les estimations initiales (Epic à 13,4 % et signée à 0,07 % par slot, jugées trop élevée et trop faible). Voir §4.
- **2026-11-08 — Clôture de l'étape 2.** Jeton API Cloudflare créé par l'utilisateur (modèle Edit Cloudflare Workers + D1, expire dans 1 an, **à renouveler vers le 2027-11-08**) et saisi dans l'environnement GitHub `production`. PR #2 fermée. Premier run « Déploiement production » réussi (tests, build, migrations, déploiement, vérification de /api/health).
- **2026-11-08 — Étape 3.** Authentification complète, détaillée au §2ter. 41 tests automatiques passent (inscription, connexion, sessions, CSRF, injection SQL, limitation, admin, règle ESLint). Parcours de bout en bout validé en local avec curl : invitation admin en ligne de commande, inscription, élévation, création d'invitation, journal, déconnexion. Vitest 5 est incompatible avec `@cloudflare/vitest-pool-workers` (qui demande vitest 4) : les tests utilisent `node:sqlite` avec les vraies migrations.
- **2026-11-08 — Correctif `admin:invitation`.** `wrangler d1 execute --remote --file` passe par l'API « import », qui refuse le jeton OAuth de `wrangler login` (Authentication error 10000). Le script utilise maintenant `--command`, qui passe par l'API « query », comme les migrations. Wrangler est lancé directement par Node, sans shell.
- **2026-11-08 — Clôture de l'étape 3.** Mot de passe minimum abaissé à 6 caractères (PR #8). Compte admin `Crousti` créé en production avec `npm run admin:invitation`, puis vérifié en base (role=admin, actif=1).
- **2026-11-08 — Étape 4.** Catalogue terminé (§3bis). 65 tests passent : classification, validation de données hostiles, cache, panne de la source, routes. Validé en local contre la vraie source : 1 407 cartes, premier chargement en 0,86 s, puis 0,05 s. Page Catalogue responsive avec filtres, miniatures WebP et fiche détaillée.
- **2026-11-08 — Correctif du catalogue vide en production.** Les journaux (`wrangler tail`) montraient les requêtes du catalogue bloquées 13,5 s sans CPU, puis annulées. Un Worker de diagnostic exécuté sur le réseau Cloudflare (`wrangler dev --remote`) a mesuré GitHub entre 3 et 9,5 s, contre 0,16 s pour Hugging Face. Le Cache API fonctionne bien sur workers.dev. Hugging Face devient la source principale, GitHub passe en secours, chaque source a 8 s au plus et la page affiche un message de chargement et d'erreur. 68 tests.
- **2026-11-08 — Favicon.** Logo de l'utilisateur (spirale orange). Le fond noir devient transparent pour l'onglet. L'icône iPhone et les icônes PWA ont le fond #0b1220. Script `outils/generer-icones.py`. Attention : ne jamais ouvrir puis sauvegarder une image dans VS Code, cela l'enregistre comme du texte et la corrompt (c'est arrivé une fois avec logo-source.png).
- **2026-11-08 — Filtre par type.** Nouveau filtre `type` (Unité, Sort, Équipement, Légende, Terrain, Rune, Token) dans l'API et sur la page. Il permet notamment de trouver les runes d'Origins, rangées en variante `base`. 70 tests.
- **2026-11-08 — Terrains et pagination.** Les Battlefields étaient déformés : ils sont maintenant tournés dans la grille et affichés à l'horizontale dans la fiche. Pagination en haut et en bas avec un choix de page. Comparaison avec la galerie officielle : il manque 2 cartes dans la source (voir §3bis). 69 tests.
- **2026-11-09 — Étape 9.** PWA et responsive (§4sexies). Service worker écrit à la main et généré au build (aucune nouvelle dépendance), `/api/*` jamais en cache. Manifeste, icône maskable, carte « Installer l'application », bandeau hors ligne et reconnexion automatique. Marges safe-area, `dvh`, réglages petits

écrans et écrans tactiles. 17 nouveaux tests (138 au total), dont l'exécution réelle du service worker dans un bac à sable. Vérifié dans Edge (installable, hors ligne, 3 tailles d'écran). Test d'échange instable corrigé (catalogue périmé après un saut de 48 h).

- **2026-11-09 — Étape 8.** Monitoring admin en quasi temps réel (§4quinquies). Migration `0005` (tirages remarquables, erreurs serveur, index). Route `GET /api/admin/monitoring` en lecture seule, qui ne prolonge pas l'accès admin. Quotas Cloudflare via l'API GraphQL (jeton à créer par l'utilisateur, §7). Onglet « Monitoring » rafraîchi toutes les 8 s. « En ligne » précis à 2 minutes. 121 tests (8 nouveaux). Vérifié de bout en bout en local.
- **2026-11-09 — Taux, filtres, images.** À la demande de l'utilisateur : (1) tous les taux Rare+ divisés par 2 (Epic 1/12, Alt 1/24, Over 1/144, Signée 1/480, Special et Ultime 1/2000 boosters), tests statistiques mis à jour ; (2) filtre Champion et Signature (étiquettes `cardTypeLabels`), fixture de test enrichie ; (3) image de secours quand une miniature du CDN ne se charge pas. 113 tests.
- **2026-11-09 — Correctif page blanche à l'ouverture.** Après l'ouverture d'un booster, cliquer sur une carte affichait une page blanche : `POST /api/boosters/ouvrir` renvoyait des cartes incomplètes (sans `domaines`, `set`, `texte`…) et la fiche plantait sur `carte.domaines.length`. L'API renvoie maintenant la carte complète du catalogue (comme la collection), plus `foil` et `slot`. Test de non-régression sur la forme des cartes. 113 tests.
- **2026-11-09 — Étape 7.** Échanges entre joueurs (§4quater). Quatre règles validées par l'utilisateur : plusieurs contre plusieurs, dons, collection complète visible, expiration à 48 h. Migration `0004`. Acceptation en un seul lot D1 atomique avec jeton de garde. 17 nouveaux tests (113 au total), dont l'atomicité et les acceptations simultanées. Vérifié de bout en bout sur le vrai moteur D1 local.
- **2026-11-09 — Étape 6.** Interface boosters et collection (§4ter). Accueil transformé en hub (stock + minuteur + choix du set + Ouvrir), animation d'ouverture en cascade avec halo et voile foil, page `/collection` (filtres, quantités, foil, doublons, complétion par set). Composants partagés extraits (`fiche-carte`, `filtres-cartes`, `pagination`). Nouvelles routes de lecture `GET /api/collection` et `/api/collection/resume`. Parcours vérifié bout-en-bout contre le vrai runtime local. 96 tests. Aucune nouvelle dépendance ni migration.
- **2026-11-09 — Étape 5.** Moteur de boosters terminé (§4bis). 3 décisions tranchées avec l'utilisateur (Légendes/Terrains dans leurs pools, doublons autorisés, foil stocké à part). Nouveau dossier `serveur-api/boosters/` (config des taux, aléatoire injectable, pools, tirage pur, stock paresseux, routes). Migration `0003` (stock sur les comptes, tables `collection` et `ouvertures_boosters`). API `/api/boosters/etat` et `/ouvrir`. 90 tests passent (dont 100 000 boosters simulés, fréquences par booster vérifiées). Migration validée sur le vrai moteur D1 en local. Un nouveau joueur reçoit 15 boosters à l'inscription.
- Astuce diagnostic : `Date.now()` n'avance pas pendant le calcul dans un Worker (protection Spectre), donc on ne peut pas mesurer le CPU ainsi. Utiliser le champ `cpuTime` de `wrangler tail --format json`.
- Astuce : PowerShell `Invoke-WebRequest` refuse les cookies `Secure` sur http://localhost (les navigateurs les acceptent). Pour tester l'API en local, utiliser `curl` avec `-b/-c` dans Git Bash.
- Astuce Bash : les heredocs contenant des apostrophes françaises échouent dans l'outil Bash de l'assistant. Écrire plutôt les scripts dans un fichier.
- Astuce Windows : `pkill` n'existe pas dans Git Bash. Pour arrêter un serveur de dev resté ouvert, utiliser PowerShell `Get-CimInstance Win32_Process -Filter "Name='node.exe'"` puis `Stop-Process`.

Arborescence (convention de nommage : français, minuscules, tirets)

```
application-web/
  index.html                          LE SITE (navigateur : téléphone, tablette, PC)
  demarrage-application.tsx           page HTML de départ (nom imposé par le web et Cloudflare)
  racine-application.tsx              démarre React
  pages/page-connexion.tsx            charge le compte connecté et choisit la page à afficher
  pages/page-inscription.tsx          formulaire de connexion
inscription#code=...)                 création de compte avec un code d'invitation (lien /
  pages/page-accueil.tsx              accueil du joueur connecté
  pages/page-admin.tsx                administration : invitations, comptes, journal
```

```
  pages/page-catalogue.tsx            catalogue : filtres, grille d'images, fiche de carte
  pages/page-collection.tsx           ma collection : cartes possédées, quantités, foil, complétion
  composants/panneau-boosters.tsx     hub : stock, minuteur, choix du set, bouton Ouvrir
  composants/ouverture-booster.tsx    animation d'ouverture des 14 cartes (cascade, halo, foil)
  composants/fiche-carte.tsx          fiche détaillée d'une carte (modale), partagée
  composants/filtres-cartes.tsx       barre de filtres partagée (catalogue + collection)
  composants/pagination.tsx           navigation entre pages de résultats, partagée
  pages/page-echanges.tsx             mes échanges : reçues, envoyées, historique ; accepter, refuser,
annuler
  pages/page-nouvel-echange.tsx       composer une proposition (joueur, cartes données et demandées,
message)
  composants/selecteur-cartes.tsx     parcourir une collection (la mienne ou celle d'un autre) et
ajouter des cartes
  composants/liste-cartes-echange.tsx liste compacte des cartes d'un échange (panier modifiable)
  composants/tableau-monitoring.tsx   tableau de bord admin (rafraîchi toutes les 8 s)
  composants/image-carte.tsx          image de carte avec repli si le CDN échoue
  outils/echanges.ts                  types, libellés des statuts, point de vue « je reçois / je donne »
  outils/libelles-cartes.ts           noms français des raretés et variantes, couleurs, URL d'images
redimensionnées
  outils/appels-api.ts                appels au serveur et messages d'erreur en français
  outils/navigation.ts                changement de page sans rechargement, format des dates
  outils/pwa.ts                       PWA : enregistrement du service worker, installation, détection
hors ligne
  composants/bandeau-hors-ligne.tsx   bandeau « Hors ligne » en haut de l'écran
  composants/carte-installation.tsx   carte « Installer l'application » (bouton natif ou mode d'emploi
iPhone)
  service-worker/modele-service-worker.js  MODÈLE du service worker (règles de cache), complété au build
  styles-globaux.css                  couleurs et mise en page communes
  fichiers-statiques/manifest.webmanifest  manifeste de l'application installable (nom, couleurs, icônes,
 raccourcis)
  fichiers-statiques/_headers         en-têtes de sécurité du site (nom imposé par Cloudflare)
  fichiers-statiques/logo-source.png  logo d'origine fourni par l'utilisateur (spirale orange)
  fichiers-statiques/favicon.ico      favicon (16, 32, 48 px), GÉNÉRÉ
  fichiers-statiques/icones/          favicon-32, apple-touch-icon (180), icone-192/512 et icone-
masquable-512 (PWA), logo transparent, GÉNÉRÉS

serveur-api/                          L'API (Cloudflare Worker, Hono)
  point-entree-api.ts                 assemble la sécurité (en-têtes, taille max, CSRF, session) et les
routes
  entetes-securite.ts                 en-têtes de sécurité des réponses de l'API
  types-application.ts                types partagés (compte connecté, rôles)
  validation-donnees.ts               règles de validation (Zod) de tout ce que le navigateur envoie
  lecture-requete.ts                  lecture et validation du corps JSON
  journal-evenements.ts               écriture dans le journal de sécurité et d'admin
  erreurs-serveur.ts                  trace des erreurs 500 pour le monitoring (purge à 7 jours)
  administration/monitoring.ts        /api/admin/monitoring : chiffres du tableau de bord
  administration/quotas-cloudflare.ts quotas gratuits du jour (API GraphQL Analytics, jeton secret)
  authentification/sessions.ts        cookie de session, chargement du compte, exigerConnexion,
exigerAdmin
  authentification/routes-authentification.ts   /api/auth : inscription, connexion, déconnexion, moi
  administration/routes-administration.ts       /api/admin : élévation, invitations, comptes, journal
  securite/mots-de-passe.ts           hachage PBKDF2-SHA256 (100 000 itérations)
  securite/jetons-et-empreintes.ts    jetons aléatoires, SHA-256, comparaison en temps constant
  securite/limitation-tentatives.ts   anti force brute (compteurs dans D1)
  securite/protection-csrf.ts         refuse les requêtes venant d'un autre site
  catalogue/types-catalogue.ts        types Carte, Set, Variante, Rareté
  catalogue/classification-cartes.ts  validation de la source et classement en variantes
  catalogue/source-cartes.ts          téléchargement et cache (mémoire, Cache API) de la source
  catalogue/routes-catalogue.ts       /api/catalogue : sets, recherche de cartes, détail
  catalogue/recherche-texte.ts        normalisation du texte (sans accents), partagée
  collection/routes-collection.ts     /api/collection : cartes possédées, résumé de complétion,
collection d'un autre joueur
  echanges/regles-echanges.ts         règles, limites et validation (Zod) des propositions
  echanges/routes-echanges.ts         /api/echanges : proposer, lister, accepter (lot atomique), refuser,
 annuler
```

```
  boosters/configuration-taux.ts      SOURCE DE VÉRITÉ des taux, composition, stock (doc §4)
  boosters/aleatoire.ts               aléatoire injectable (crypto en prod, PRNG seedé pour les tests)
  boosters/pools.ts                   pré-indexe les cartes par set/rareté/variante (mémoïsé)
  boosters/tirage-booster.ts          tirage PUR des 14 cartes selon les taux
  boosters/stock.ts                   stock paresseux (+1/10 min, max 15)
  boosters/routes-boosters.ts         /api/boosters : etat, ouvrir
  tests/                              tests automatiques (vraie base SQLite en mémoire + vraies
migrations)
    outils/base-d1-de-test.ts         imite D1 avec node:sqlite
    outils/client-api-de-test.ts      appelle l'API comme un navigateur
    outils/catalogue-de-test.ts       catalogue synthétique pour les tests de boosters
    pwa.test.ts                       service worker exécuté dans un bac à sable, manifeste, icônes, en-
têtes
    donnees/cartes-echantillon.json   47 vraies cartes (une par set et par catégorie)
  types-cloudflare-generes.d.ts       GÉNÉRÉ par `npm run cf-typegen`, ne pas modifier à la main

base-de-donnees/                      BASE SQLITE CLOUDFLARE D1
  migrations/0001_initialisation.sql  table parametres
  migrations/0002_comptes_et_sessions.sql  comptes, sessions, invitations, limitations, journal
  migrations/0003_boosters_et_collection.sql  stock de boosters, collection, ouvertures
  migrations/0004_echanges.sql        échanges et cartes des échanges
  migrations/0005_monitoring.sql      tirages remarquables, erreurs serveur, index du monitoring
  (les migrations sont numérotées et jamais modifiées après le merge : on en ajoute une nouvelle)

configuration/                        RÉGLAGES DES OUTILS
  cloudflare-wrangler.jsonc           Cloudflare : nom du service, règles de routage, base D1
  vite-site-et-api.config.ts          Vite : `npm run dev` et `npm run build`
  plugin-service-worker.ts            Vite : fabrique service-worker.js au build (fichiers à précharger,
version)
  tests-vitest.config.ts              Vitest : où sont les tests
  qualite-code-eslint.config.js       ESLint : règles de qualité et de sécurité du code
  typescript/tsconfig.json            TypeScript : fichier principal, qui renvoie aux 4 suivants
  typescript/tsconfig-application-web.json
  typescript/tsconfig-serveur-api.json
  typescript/tsconfig-configuration.json
  typescript/tsconfig-tests.json

documentation/                        REPRISE DU PROJET
  demande-initiale.md                 ta demande d'origine, mot pour mot
  etat-du-projet.md                   CE FICHIER : décisions, étape en cours, feuille de route
  rapports-avancement/*.pdf           un rapport PDF par étape

outils/
  generer-rapport-pdf.py              fabrique le PDF de fin d'étape
  creer-invitation-admin.mjs          SEUL moyen de créer un compte admin (`npm run admin:invitation`)
  generer-icones.py                   régénère favicon et icônes depuis logo-source.png (`python outils/
generer-icones.py`)

.github/                              GITHUB (emplacement imposé)
  workflows/verifications-qualite.yml lint, types, tests, build, audit à chaque PR
  workflows/analyses-securite.yml     CodeQL et Gitleaks à chaque PR et chaque lundi
  workflows/deploiement-production.yml  mise en ligne automatique après merge sur main
  dependabot.yml                      mises à jour automatiques des dépendances (nom imposé)
  SECURITY.md                         politique de sécurité (nom imposé)

À la racine (emplacement imposé par les outils) :
  package.json / package-lock.json    dépendances et commandes npm
  .npmrc                              réglages npm (versions exactes)
  .gitignore / .gitattributes         ce que Git ignore, fins de ligne
  README.md                           page d'accueil GitHub
  CLAUDE.md                           consignes lues par l'assistant à chaque nouvelle conversation
```